

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

WARNING

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the *Copyright Act 1968* (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.



Acknowledgements

This material includes content adapted from instructional resources made available by David Silver as part of his Reinforcement Learning course at UCL under CC-BY-NC 4.0.

Refer to <https://www.davidsilver.uk/teaching/> for full details.



FIT5226

Multi-agent System & Collective Behaviour

Wk 6: MC Control, SARSA and Q-Learning

Week-to-week structure (tentative)

	1	Introduction & Scoping	
	2	Macroscopic/mesoscopic system dynamics (deterministic & stochastic)	
Learning ↑	3	RL Intro and MDPs	Bellman Equations
	4	Policy and Value Iteration	SARSA
	5	Q-Learning	Deep Q-Learning
	6	Game Theory Intro	
	7	Solution Concepts; Nash	MiniMax-Q, Markov Games
Multi-agent ↑	8	Population Games	Evolutionary Game Theory
	9	Social Dilemmas	
	10	Coalition Game Theory	
	11	Auctions	
	12	Voting	

From Model-based Prediction to Model-free Control

So far:

Model-based prediction and control

Estimate/Optimise the value function of a *known* MDP (given P and R)

$$v'(s) \rightarrow \max_{a \in A} R_s^a - \gamma \sum_{s' \in S} P_{s,s'}^a v_{\star}^{s'}$$

The goal:

Model-free control

Optimise the value function of an *unknown* MDP

Step 1: From Model-based Prediction to Model-free-Prediction

To achieve model-free prediction (value estimation) we first need to get rid of the transition matrix that we use in model-based prediction to compute the expectation.

$$v'(s) \rightarrow \max_{a \in A} R_s^a + \gamma \sum_{s' \in S} P_{s,s'}^a v_{\star}^{s'}$$

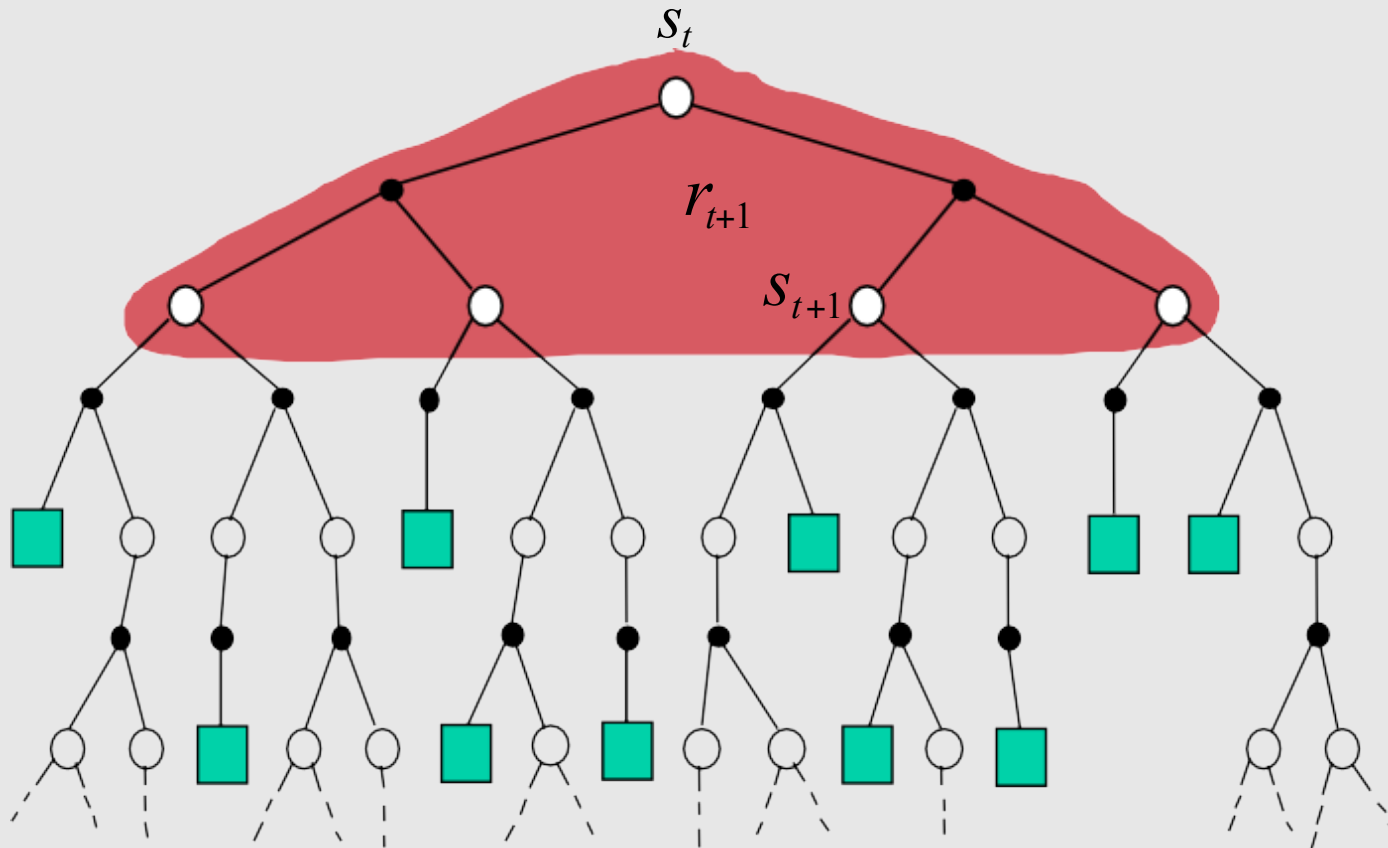
We can use sampling instead.

In this way we will arrive at model-free estimation.

Sampling liberates us from needing a model

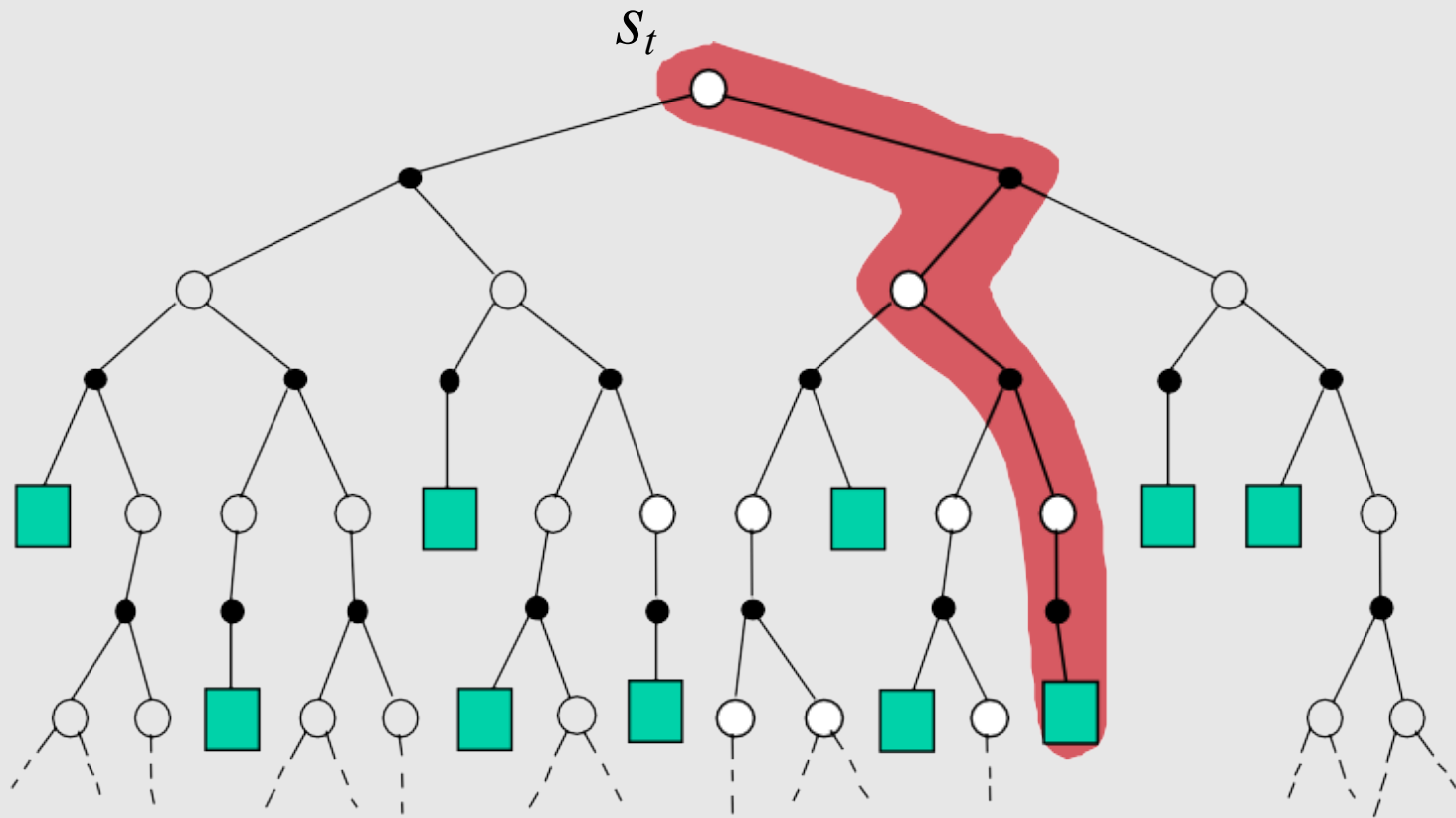
replace the expectation by an (incrementally computed) sample-mean

$$V(S_t) = \mathbb{E}[R_{t+1} + \gamma V(S_{t+1})]$$



Monte-Carlo Update

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$



RL Terminology: the structure of the update is called a “backup”

Monte-Carlo Policy Evaluation

Goal: learn v_π from episodes of experience under policy π

$$S_1, A_1, R_2, \dots, S_k \sim \pi$$

Recall that the return is the total discounted reward:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-1} R_T$$

Recall that the value function is the expectation of the return:

$$v_\pi(s) = \mathbb{E} [G_t \mid S_t = s]$$

Monte-Carlo policy evaluation uses
empirical mean return instead of expected return

Every-Visit Monte-Carlo Policy Evaluation

To evaluate state s

- **Every** time-step t that state s is visited in an episode, Increment counter $N(s) \leftarrow N(s) + 1$
- Increment total return $S(s) \leftarrow S(s) + G_t$
- Value is estimated by mean return $V(s) \leftarrow S(s)/N(s)$
- $\lim_{N(s) \rightarrow \infty} V(s) = v_{\pi}(s)$

Incremental Monte-Carlo Updates

Update the empirical mean of $V(s)$ **incrementally** after episode $S_1, A_1, R_2, \dots, S_T$

For each state S_t with return G_t

$$V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)} ((G_t - V(S_t)))$$

In non-stationary problems, it can be useful to track a running mean, i.e. forget old episodes:

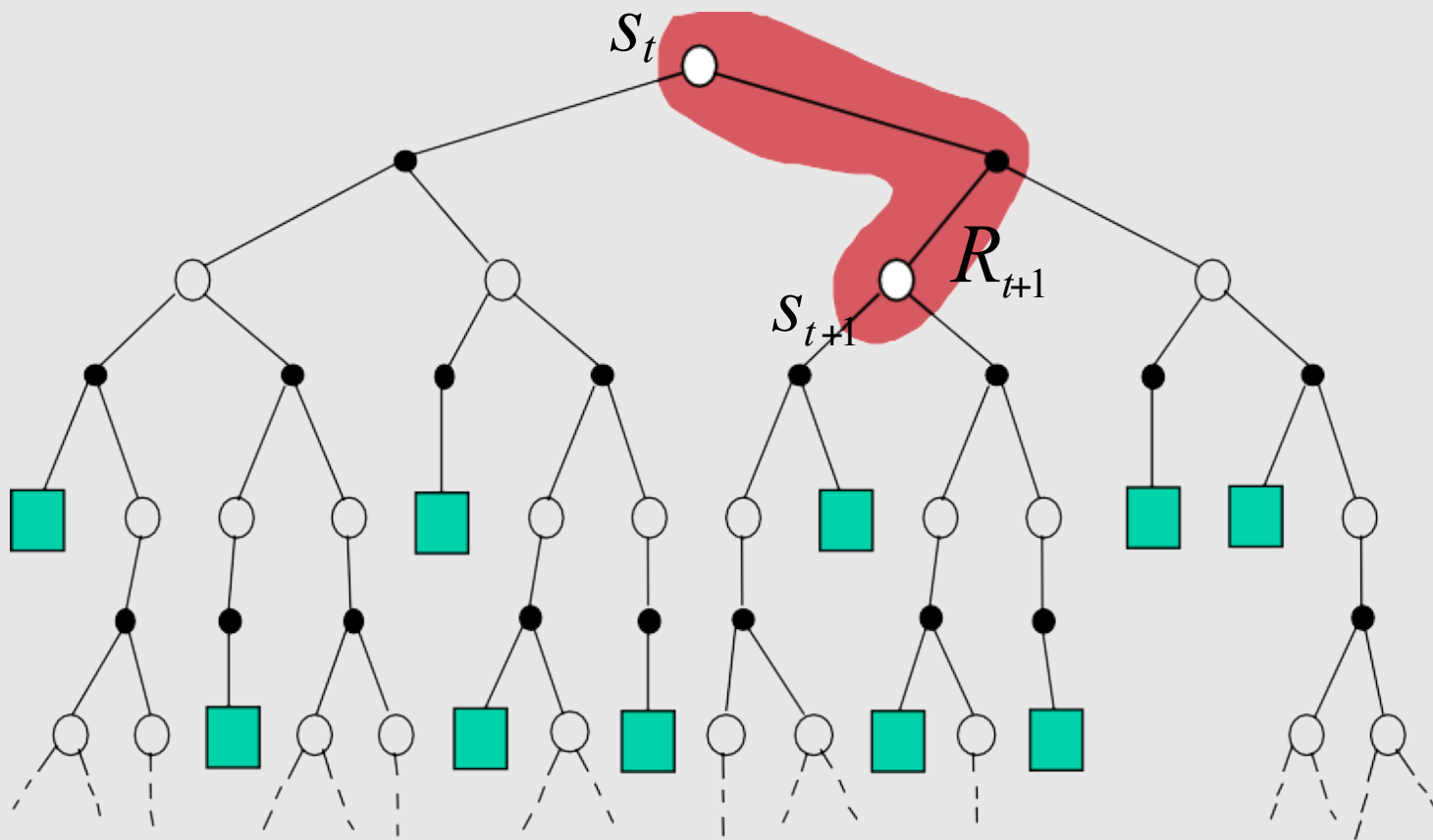
$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

Monte-Carlo Reinforcement Learning

- MC methods learn directly from episodes of experience
- MC is model-free: no knowledge of MDP transitions or rewards
- MC learns from complete episodes: no bootstrapping
- MC uses the simplest possible idea: $\text{value} = \text{sample mean return}$
- **Caveat:**
 - **All episodes must terminate**
 - **MC can only be applied to episodic MDPs**

Temporal-Difference Update

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$



Monte-Carlo versus Temporal Difference

Goal: learn V_π online from experience under policy π

- Incremental every-visit **Monte-Carlo**

- Update value $V(S_t)$ **toward actual return G_t**

$$V(S_t) \leftarrow V(S_t) + \alpha(G_t - V(S_t))$$

- Simplest **temporal-difference** learning algorithm: TD(0)

- Update value $V(S_t)$ **toward estimated return $R_{t+1} + \gamma V(S_{t+1})$**

$$V(S_t) \leftarrow V(S_t) + \alpha(R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

- $R_{t+1} + \gamma V(S_{t+1})$ is called the **TD target**; replacing the empirical return G
 - $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the TD error

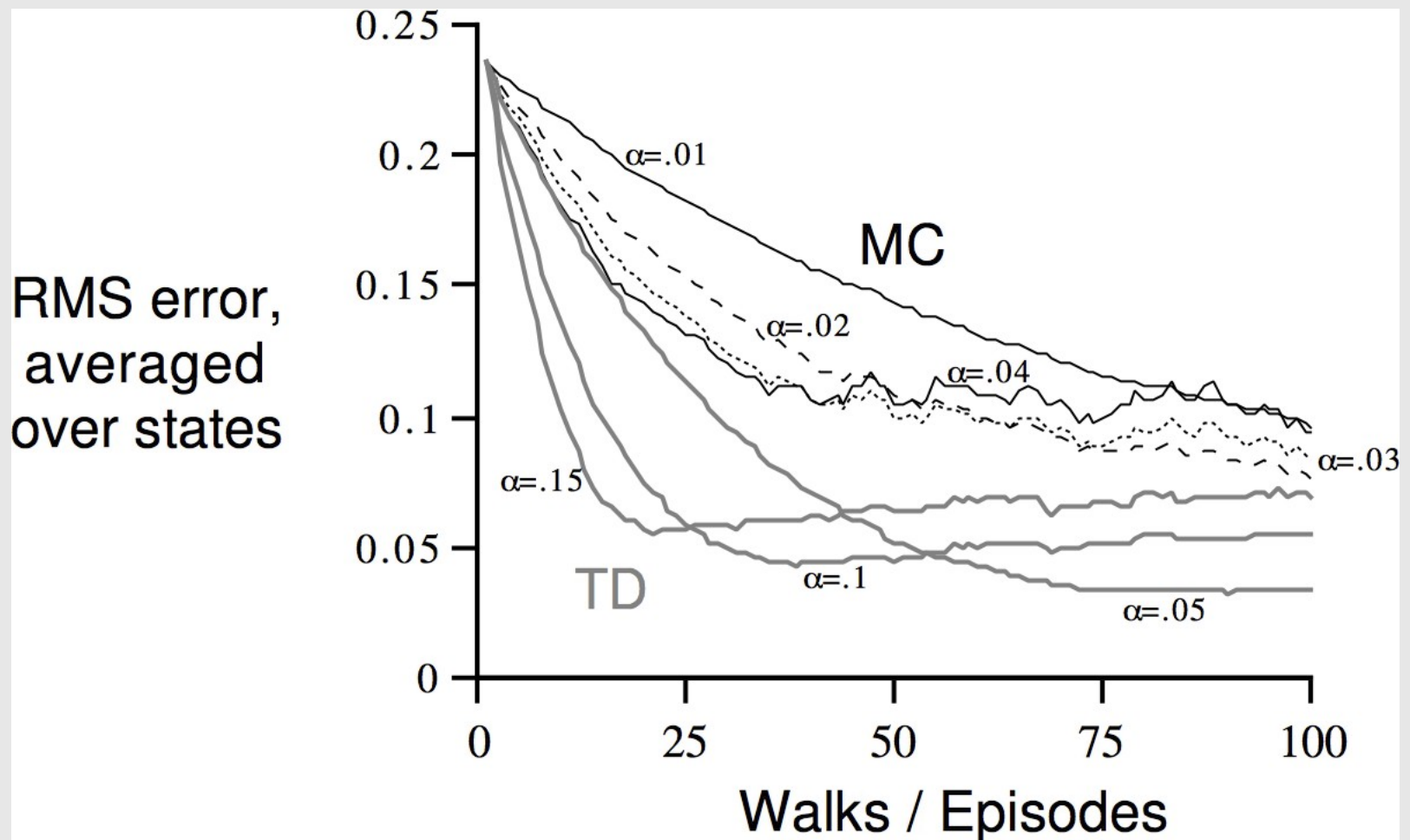
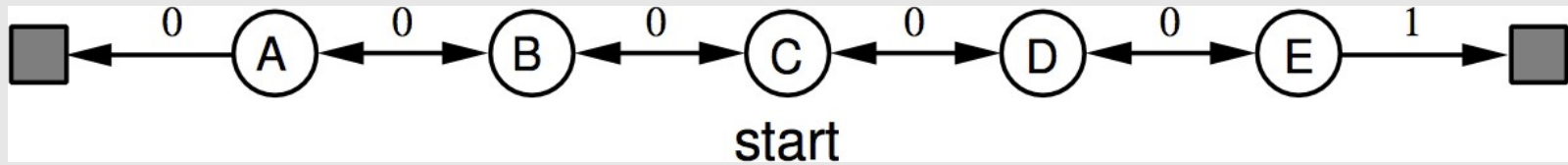
Advantages and Disadvantages of MC vs. TD

- TD can learn before and without the final outcome
- TD can learn online after every step
- MC must wait until end of episode before return is known
- TD can learn from incomplete sequences MC can only learn from complete sequences
- TD works in continuing (non-terminating) environments
- MC only works for episodic (terminating) environments

Advantages and Disadvantages of MC vs. TD (cont'd)

- MC has high variance, zero bias
 - good convergence properties
 - not very sensitive to initial value
 - very simple to understand and use
- TD has low variance, some bias
 - usually more efficient than MC
 - TD converges to $v_{\pi}(s)$ (with value tables)
 - more sensitive to initial value

Random Walk: MC vs. TD



Step 2: From Model-free Prediction to Control

Ultimately, we need to find the optimal policy π

So far we have worked with the state-value function $V(s)$

Greedy policy improvement over $V(s)$ requires model of MDP

$$\pi'(s) = \operatorname{argmax}_{a \in A} R_s^a + P_{s,s'}^a V(s')$$

We can instead directly work with the action-value function.

Greedy policy improvement over $Q(s, a)$ is model-free

$$\pi'(s) = \operatorname{argmax}_{a \in A} Q(s, a)$$

But now we need to find Q instead of V !

Monte-Carlo Policy Iteration (simple)

1. **Policy evaluation** Monte-Carlo policy evaluation (as above **but for q**),

$$Q \approx q_\pi$$

2. **Policy improvement** ϵ -greedy policy improvement

with probability $(1 - \epsilon)$ choose $\operatorname{argmax}_{a \in A} Q(S, a)$;

with probability ϵ choose randomly

3. repeat

There are alternative ways to handle the policy improvement but they must be **GLIE**

GLIE Monte-Carlo Control (Greedy in the limit with infinite exploration)

Sample k -th episode using current $\pi : \{S_1, A_1, R_2, \dots, R_T\} \sim \pi$

For each state S_t and action A_t in the episode

$$N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$$
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - Q(S_t, A_t))$$

Improve policy based on new action-value function

$\epsilon \leftarrow 1/k$ $\longleftarrow$ Infinite Exploration

$\pi \leftarrow \epsilon\text{-greedy}(Q)$ $\longleftarrow$ Greedy in the limit

Theorem

GLIE Monte-Carlo control converges to the optimal state action-value function
 $Q(s, a) \rightarrow q_\star(s, a)$

MC vs. TD for Control

Temporal-difference (TD) learning has several advantages over Monte-Carlo (MC)

- can use incomplete sequences
- lower variance

Thus we expect improvement by using TD instead of MC in the control loop

- Apply TD to $Q(S,A)$
- ϵ -greedy policy improvement
- update every time-step

Policy Iteration with SARSA

Every time step

1. **Policy evaluation update** for state-action value using SARSA

2. **Policy improvement** ϵ -greedy policy improvement

with probability $(1 - \epsilon)$ choose $\operatorname{argmax}_{a \in A} Q(S, a)$;

with probability ϵ choose randomly

3. repeat

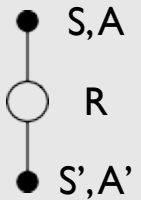
$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

Summary: SARSA - TD Backups

We only consider sample TD-backups

Using sample rewards and sample transitions

(S, A, R, S', A')



Note, you do need to determine the action A' in the second state S'

We do **not** need the reward function R and transition dynamics P

Advantages:

- Model-free: no advance knowledge of MDP required
- Breaks the curse of dimensionality through sampling
- Cost of an individual backup is constant, independent of $n = |S|$

Sarsa Algorithm for On-Policy Control

Every time-step

- Policy evaluation SARSA $Q \approx q_\pi$
- Policy improvement ϵ -greedy

Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$

Repeat (for each episode):

 Initialize S

 Choose A from S using policy derived from Q (e.g., ϵ -greedy)

 Repeat (for each step of episode):

 Take action A , observe R, S'

 Choose A' from S' using policy derived from Q (e.g., ϵ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

 until S is terminal

Convergence of SARSA

Theorem

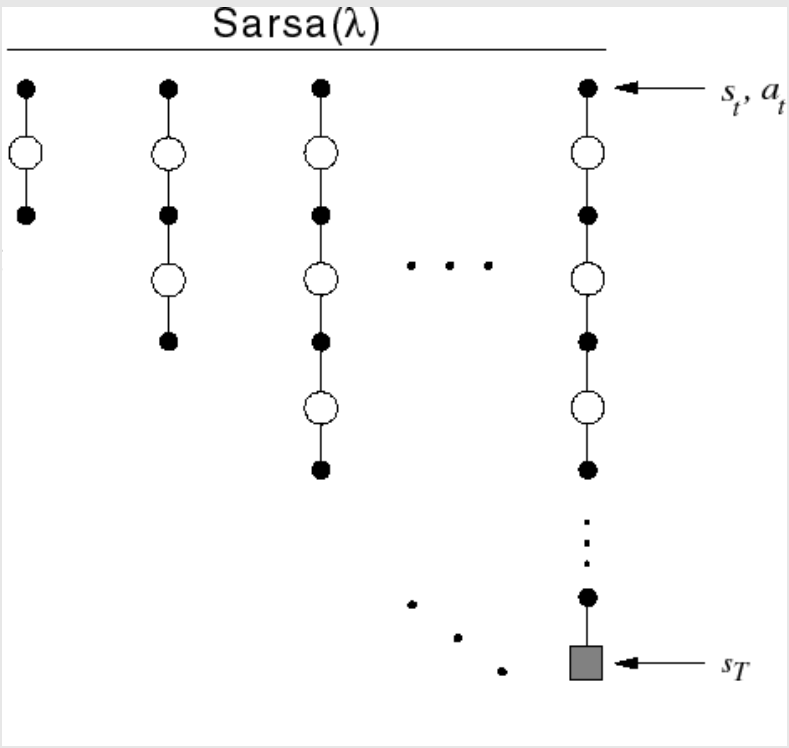
Sarsa converges to the optimal action-value function $Q(s, a) \rightarrow q_{\star}(s, a)$, under the following conditions:

- GLIE sequence of policies $\pi_t(a \mid s)$
- Robbins-Monro sequence of step-sizes α_t

- $\sum_{t=1}^{\infty} \alpha_t = \infty$

- $\sum_{t=1}^{\infty} \alpha_t^2 < \infty$

n-step SARSA



n=1 Sarsa $q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1})$

n=2 $q_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2})$

...

n= ∞ MC $q_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$

n-step Sarsa

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^{(n)} - Q(S_t, A_t) \right)$$

n -step SARSA Gridworld Example [Sutton Barto]

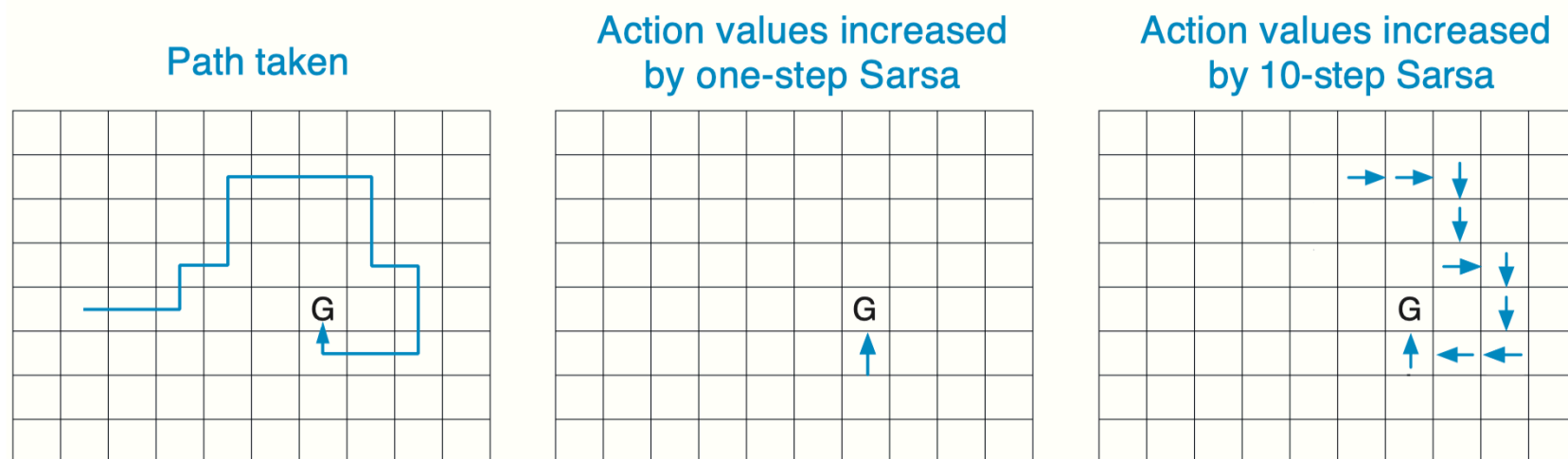


Figure 7.4: Gridworld example of the speedup of policy learning due to the use of n -step methods. The first panel shows the path taken by an agent in a single episode, ending at a location of high reward, marked by the G. In this example the values were all initially 0, and all rewards were zero except for a positive reward at G. The arrows in the other two panels show which action values were strengthened as a result of this path by one-step and n -step Sarsa methods. The one-step method strengthens only the last action of the sequence of actions that led to the high reward, whereas the n -step method strengthens the last n actions of the sequence, so that much more is learned from the one episode.

Backward View TD(λ)

- Forward view (above) provides theory
- Backward view provides implementation mechanism
 - updates online, every step, from incomplete sequences
- In practice, you would use SARSA(λ), a backwards methods that assigns credit to past actions
 - backwards view can be adjusted in length between
 - $\lambda=0$: pure SARSA
 - $\lambda=1$: credit for all past actions in the episode

On-Policy vs Off-Policy Learning

On-policy learning

- *Learn on the job*
- Learn about policy π from experience sampled from π
- i.e. sample according to the policy you are learning
- e.g. SARSA

Off-policy learning

- *Look over someone's shoulder*
- Learn about policy π from experience sampled from μ

Off-Policy Learning

Evaluate **target policy** $\pi(a \mid s)$ to compute $v_\pi(s)$ or $q_\pi(s, a)$
while following **behaviour policy** $\mu(a \mid s)$

$$\{S_1, A_1, R_2, \dots, S_T\} \sim \mu$$

Why is this important or useful?

- **Learn about optimal policy following exploratory policy**
- Learn from observing humans or other agents
- Re-use experience generated from old policies

$$\pi_1, \pi_2, \dots, \pi_{t-1}$$

Q-Learning

We now consider off-policy learning of action-values $Q(s, a)$

- the next action is chosen using **behaviour** policy

$$A_t \sim \mu(\cdot \mid S_t) \text{ typically } \epsilon\text{-greedy w.r.t. } Q$$

A_t determines the next state S_{t+1} and reward R_{t+1}

- we then consider a successor action from the **target** policy

$$A^\pi \sim \pi(\cdot \mid S_{t+1}) \text{ typically greedy w.r.t. } Q$$

we determine the state value of this next state S_{t+1} from its Q -values by choosing an action A^π according to the **target policy**

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma Q(S_{t+1}, A^\pi) - Q(S_t, A_t))$$

Q-Learning Control

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right)$$

Theorem

Q-learning control converges to the optimal action-value function,
 $Q(s, a) \rightarrow q_{\star}(s, a)$

Q-Learning Algorithm for Off-Policy Control

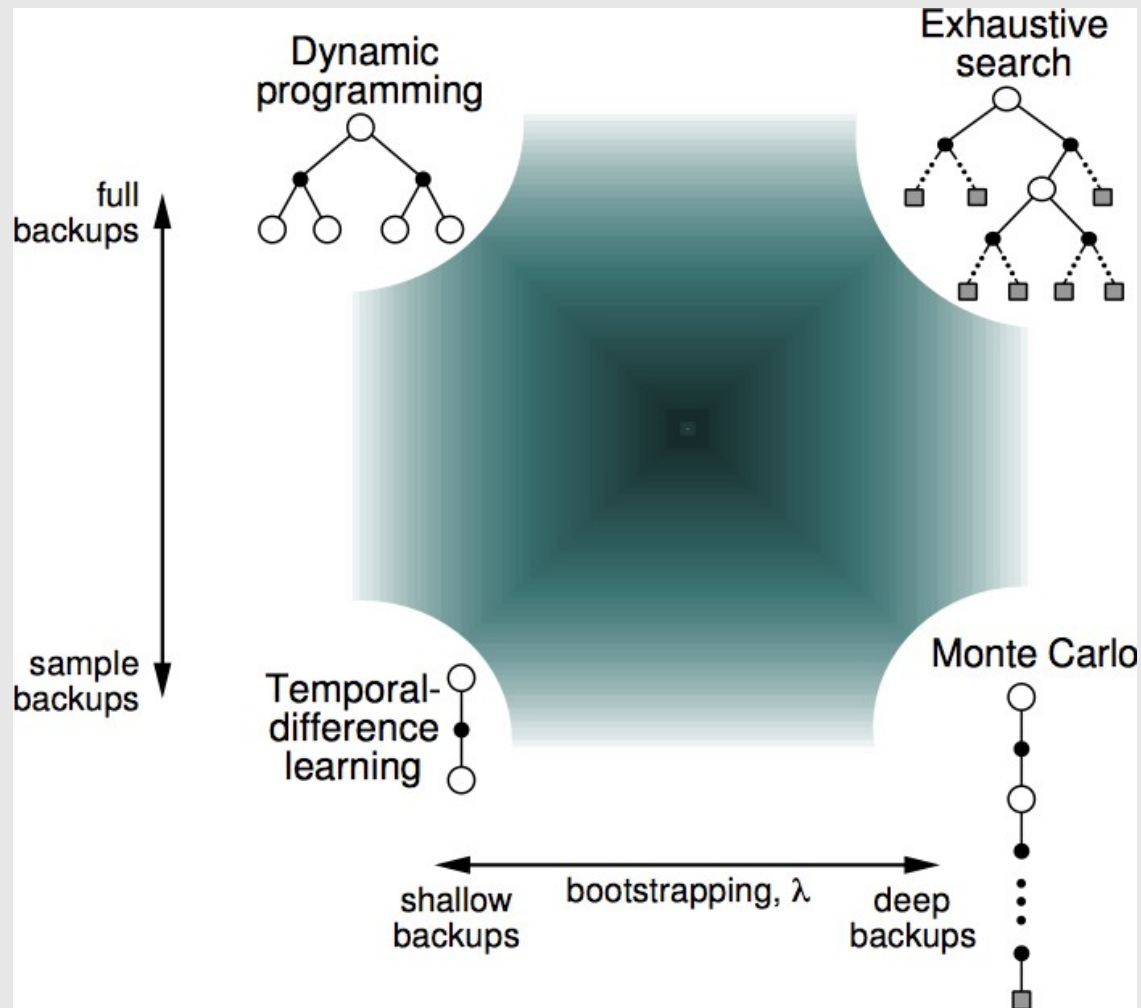
Initialize $Q(s, a), \forall s \in \mathcal{S}, a \in \mathcal{A}(s)$, arbitrarily, and $Q(\text{terminal-state}, \cdot) = 0$
Repeat (for each episode):
 Initialize S
 Repeat (for each step of episode):
 Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 Take action A , observe R, S'
 $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$
 $S \leftarrow S'$
 until S is terminal

Relationship Between DP and TD

	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>	<i>Update</i>
Bellman Expectation Equation for $v_{\pi}(s)$	Policy Evaluation	TD Learning	$V(S) \stackrel{\alpha}{\leftarrow} R + \gamma V(S')$
Bellman Expectation Equation for $q_{\pi}(s, a)$	Policy Iteration	SARSA	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma Q(S', A')$
Bellman Optimality Equation for $q_*(s, a)$		Q-Learning	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma \max_{a' \in A} Q(S', a')$

where $x \stackrel{\alpha}{\leftarrow} y$ is $x \leftarrow x + \alpha(y - x)$

Unified View of Reinforcement Learning



Take home lessons

- To work without a model with resort to sampling
- We can either learn from samples of full episodes (MC) or at every step (TD)
- MC methods have higher variance but no bias.
- TD is usually more efficient but sensitive to initial values
- To guarantee convergence sampling (usually) needs to be GLIE
- On-policy methods use the same policy they are learning to sample; off-policy methods can use a sampling policy that is different from the one being learned
- SARSA and Q-Learning are simple TD methods to learn the q function
- SARSA is on-policy; Q-Learning is off-policy